EX. NO: 5 Implementations of Ontology with FOL

Date :15.09.2025

Aim:

To implement a simple First Order Logic with Ontology Program.

Procedure :

To Create a First-Order Logic + Ontology Program in Python

Step 1: Define Your Ontology Components

Identify Classes (concepts) — e.g., Human, Animal, Mortal.

Identify Individuals (instances) — e.g., Socrates, Doggo.

Identify Relationships between individuals — e.g., hasPet.

Step 2: Represent Ontology in Python Data Structures

Use dictionaries or sets to hold classes and their members.

Use a dictionary to map individuals to their classes.

Use nested dictionaries for relationships.

Step 3: Define First-Order Logic Rules as Python Functions

Each rule represents a logical statement you want to enforce.

Example:Rule: *All humans are mortal* → For every individual who is a Human, add them to Mortal.

Step 4: Implement Reasoning Procedure

Write a main function that:

Prints initial ontology info.

Applies inference rules.

Prints results after reasoning.

Displays relationships.

Step 5: Run and Test

Execute the program.

Verify that the inference works (e.g., Socrates inferred as Mortal).

Check the output matches expectati

Step 6 (Optional): Extend Ontology or Add More Rules

Add more classes, individuals, and relationships.

Implement more complex rules, e.g., "If x has a pet y, and y is an animal, then..."

Use libraries like owlready2 for richer ontologies and reasoning support.

Program Coding:

```
# Ontology
classes = {
    "Human": set(),
    "Animal": set(),
    "Mortal": set()
}
```

```
individuals = {
    "Socrates": "Human",
    "Doggo": "Animal"
}

relationships = {
    "hasPet": {
        "Socrates": "Doggo"
    }
}

# First-order logic rules (implemented as functions)
# Rule 1: All humans are mortal
def infer_mortality():
    for person, person_type in individuals.items():
        if person_type == "Human":
            classes["Mortal"].add(person)
# Rule 2: If a human has a pet, the pet is an animal (already known in individuals)
# Rule 3: Print relationships
```

```
def print_relationships():
    print("\n--- Relationships ---")
    for subject, obj in relationships["hasPet"].items():
        print(f"{subject} has a pet named {obj}.")
# Reasoning
def main():
    print("--- Ontology Classes ---")
    for cls in classes:
        print(f"{cls}: {classes[cls]}")


    print("\n--- Individuals ---")
    for ind, cls in individuals.items():
        print(f"{ind} is a {cls}")


    # Apply rule: all humans are mortal
    infer_mortality()
    print("\n--- After Reasoning ---")
    print("Mortal:", classes["Mortal"])


    # Show relationships
```

```
    print_relationships()


# Run the program
if __name__ == "__main__":
    main()
```

Output:

```
--- Ontology Classes ---
Human: set()
Animal: set()
Mortal: set()

--- Individuals ---
Socrates is a Human
Doggo is a Animal

--- After Reasoning ---
Mortal: {'Socrates'}
```

--- Relationships ---

Socrates has a pet named Doggo.

Result:

The program successfully inferred that Socrates is mortal based on the rule "All humans are mortal" and displayed the existing relationship that Socrates has a pet named Doggo.